

IVI In-Vehicle Infotainment Systems

Conceptual Architecture of an IVI System Integrating
Media, Navigation and Projection Features

Project Report

Submitted by: Kuntal Pradhan

Roll No.: 26

Department: ECE

Institution: Institute of Engineering and Management Kolkata

Date: 29th September 2026

Table of Contents

1. Abstract
2. Introduction and Problem Statement
3. Objectives and Scope
4. System Architecture Overview
5. Layer-by-Layer Description
6. Interface Specification
7. Data Flow Scenarios
8. Audio Focus and Interruption Handling
9. Design Decisions and Rationale
10. Tools and Technologies
11. Validation Approach
12. Limitations and Future Work
13. Conclusion
14. References

1. Abstract

An In-Vehicle Infotainment (IVI) system is the software and hardware platform that delivers media, navigation, communication and connected services to vehicle occupants. This report presents a conceptual architecture for an IVI system that integrates three feature domains: media playback, turn-by-turn navigation, and smartphone projection (Android Auto / Apple CarPlay style). The architecture follows a five-layer model inspired by Android Automotive OS: user input, applications, framework services, hardware abstraction layer (HAL) and hardware. Every inter-layer boundary is annotated with the interface used (Binder/AIDL, HAL interfaces, PCM audio, CAN/NMEA, USB/Wi-Fi). The report also describes end-to-end data flows for representative scenarios and explains how audio focus arbitration resolves conflicts between media, navigation prompts and phone calls.

2. Introduction and Problem Statement

Modern vehicles ship with a central headunit that combines many functions that were once separate devices: radio, music player, GPS navigator and hands-free phone. These functions share scarce resources such as the display, speakers and processor, and they must behave predictably: a navigation prompt must be heard over music, and an incoming call must pause media and resume it afterwards. Safety-related constraints also apply because driver distraction must be minimised.

The problem addressed here is to define a clear, layered architecture that shows how these features are integrated, which components own which responsibilities, how the components talk to each other, and how a user action travels from the touchscreen to the speakers or the map.

3. Objectives and Scope

- Design a conceptual architecture diagram of an IVI system covering media, navigation and projection.
- Annotate the interface used at every component boundary.
- Describe the data flow for key scenarios, from user input to hardware output.
- Explain audio focus arbitration and interruption handling.
- Keep the design implementation-agnostic but aligned with Android Automotive terminology.

In scope: conceptual design, interfaces, data flow and behaviour rules. **Out of scope:** production code, vehicle-specific hardware bring-up, cybersecurity certification and ISO 26262 functional-safety analysis (mentioned only as future work).

4. System Architecture Overview

The system is organised into five horizontal layers. Requests flow downward (user to hardware); sensor and media data flow upward or sideways. Each of the three feature domains occupies a vertical column that crosses all layers, so a fault or change in one domain has limited impact on the others.

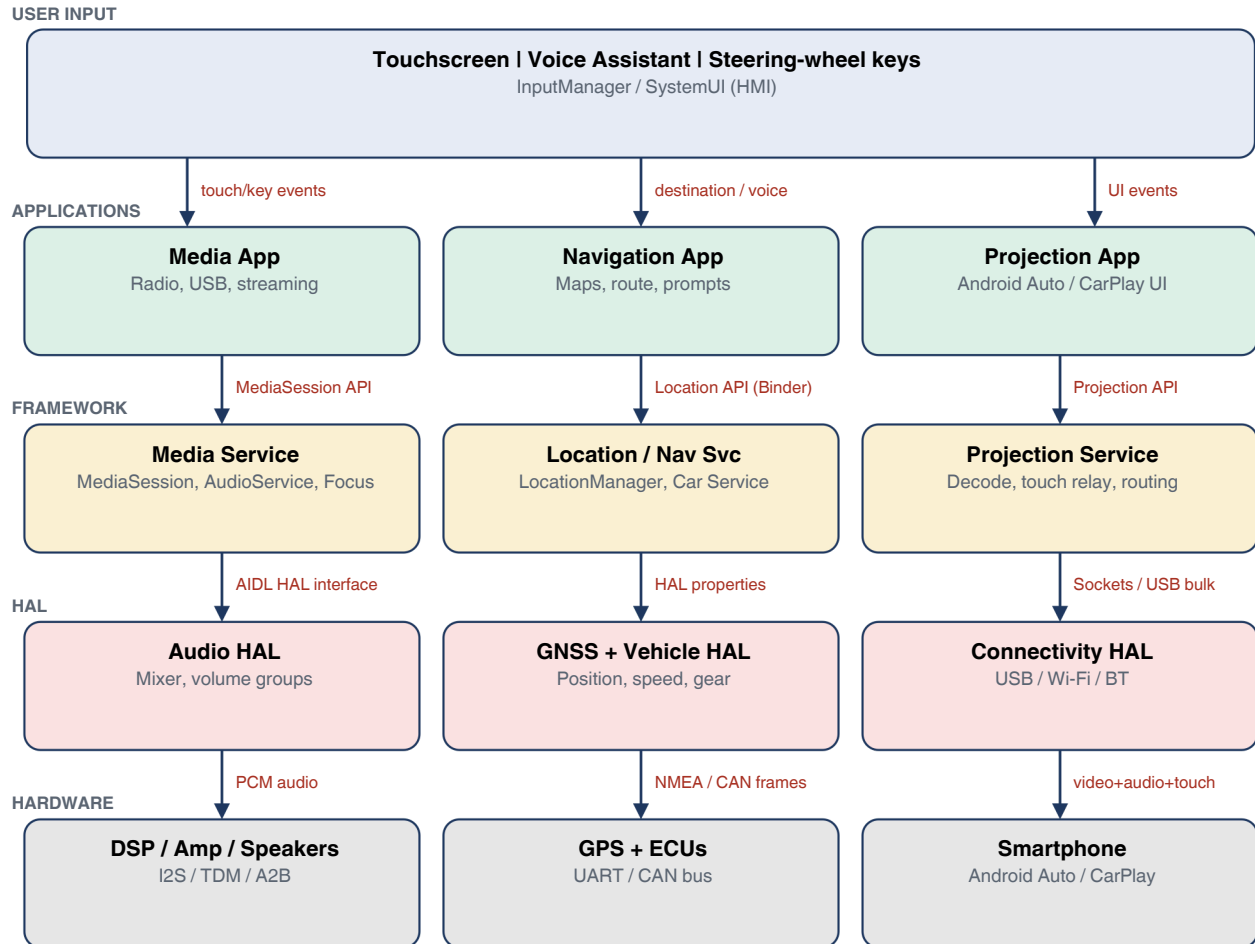


Figure 1: Five-layer IVI architecture. Red labels show the interface on each arrow. Audio focus arbitration in the Framework layer coordinates the media, navigation and projection columns.

Display rendering is shared by all applications: each app draws to a surface, SurfaceFlinger composes the surfaces, and the display controller drives the head-unit screen.

5. Layer-by-Layer Description

5.1 User Input Layer

Touchscreen taps, voice commands and steering-wheel key presses are captured by the input subsystem and delivered as events to the foreground application. Voice requests are handled by the voice assistant, which forwards intents (for example, "navigate home") to the relevant application.

5.2 Application Layer

- **Media App:** presents sources such as FM/DAB radio, USB storage, Bluetooth audio and online streaming. It publishes playback state and metadata through a media session.
- **Navigation App:** computes routes, renders maps, and issues spoken turn instructions. It consumes location updates and vehicle speed.
- **Projection App:** shows the phone's projected user interface. It forwards touch events to the phone and displays the video stream received from it.

5.3 Framework / Services Layer

- **Media Service:** manages media sessions, browsing, transport controls (play, pause, next) and works with AudioService for volume and focus.
- **Location / Navigation Services:** fuse GNSS position with vehicle speed and heading to provide stable location updates to applications.
- **Projection Service:** negotiates the projection connection, decodes the incoming video, relays touch input and routes phone audio into the car audio system.

5.4 Hardware Abstraction Layer (HAL)

The HAL hides hardware differences from the framework. The **Audio HAL** exposes mixing, zones and volume groups. The **GNSS HAL** delivers position fixes and the **Vehicle HAL** exposes vehicle properties such as speed and gear as typed key-value properties. The **Connectivity HAL** covers USB (accessory mode), Wi-Fi and Bluetooth transports used by projection and calls.

5.5 Hardware Layer

The DSP, amplifier and speakers are fed by digital audio buses (I2S, TDM or A2B). The GPS receiver and vehicle ECUs are reached over UART and CAN. The smartphone connects over USB or wireless links and runs the projected application.

6. Interface Specification

The table lists every boundary in the architecture together with the interface, the data exchanged and the direction of flow.

Boundary	Interface / Protocol	Data exchanged	Direction
Input to Apps	Input events (InputManager)	Touch coordinates, key codes, voice intents	Down
Media App to Media Service	MediaSession API over Binder	Transport commands, metadata, playback state	Both
Nav App to Location Service	Location API over Binder/AIDL	Location fixes, speed, heading, route requests	Both
Projection App to Projection Service	Projection API over Binder	UI events, decoded frames, touch relay	Both
Media Service to Audio HAL	AIDL/HIDL HAL interface	Audio streams, routing, volume group settings	Down
Location Service to GNSS/Vehicle HAL	HAL properties (AIDL)	Position, speed, gear properties	Up
Projection Service to Connectivity HAL	Sockets, USB bulk transfer	Video, audio and touch streams	Both
Audio HAL to DSP/Amp	I2S / TDM / A2B	PCM audio samples	Down
GNSS/Vehicle HAL to GPS/ECU	UART (NMEA) / CAN frames	Sentences and signal frames	Up
Connectivity HAL to Phone	USB accessory / Wi-Fi	Encoded video, audio, touch	Both

7. Data Flow Scenarios

7.1 Scenario A: User plays a song

Step	From	To	Action
1	Touchscreen	Media App	Tap on Play; event delivered via InputManager
2	Media App	Media Service	Transport control play() through MediaSession; audio focus requested
3	Media Service	Audio HAL	Stream routed to the media volume group and zone
4	Audio HAL	DSP / Speakers	PCM data sent over I2S/TDM and amplified

7.2 Scenario B: Navigation guidance

Step	From	To	Action
1	GPS receiver	GNSS HAL	NMEA sentences over UART
2	ECU	Vehicle HAL	Speed and heading via CAN
3	HALs	Location Service	Fused location published to subscribers
4	Location Service	Navigation App	Map is updated and next maneuver computed
5	Navigation App	Audio path	Voice prompt plays with transient focus; media volume is ducked

7.3 Scenario C: Phone projection

Step	From	To	Action
1	Phone	Connectivity HAL	USB/Wi-Fi link established and authenticated
2	Connectivity HAL	Projection Service	Video, audio and control channels opened
3	Projection Service	Projection App	Decoded frames rendered on the head-unit display
4	Touchscreen	Phone	Touch events relayed back through the same stack
5	Phone audio	Audio HAL	Projected audio mixed through the car audio system

8. Audio Focus and Interruption Handling

Because media, navigation prompts, projected audio and calls compete for the same speakers, the framework applies audio focus rules. An application requests focus of a specific type; the system grants it and notifies the previous holder, which then pauses, ducks or stops.

Event	Requester	Effect on current audio	After event ends
Navigation prompt	Navigation App (transient, may duck)	Media volume lowered (ducked)	Media returns to full volume
Incoming phone call	Telephony (transient, exclusive)	Media and navigation prompts paused/muted	Media resumes automatically
Projected app starts audio	Projection Service	Local media paused	Local media may be resumed by the user
Voice assistant	Assistant (transient)	Media ducked or paused	Media restored

This behaviour is what allows a navigation session to continue while music plays, and allows media to resume automatically after a call ends. Priority order, from highest to lowest, is: emergency and safety alerts, phone call, navigation prompt, voice assistant, media.

9. Design Decisions and Rationale

Decision	Rationale
Layered architecture	Separates UI, logic and hardware so each can change independently
HAL abstraction	Lets the same framework run on different SoCs, amplifiers and vehicles
Vertical feature columns	Isolates faults; a projection failure should not stop navigation or media
Central audio focus arbitration	Single place to enforce priority and avoid overlapping audio
Binder/AIDL for service calls	Well-defined, versionable, permission-checked inter-process interfaces
Vehicle HAL properties	Typed properties give apps safe, uniform access to vehicle data

10. Tools and Technologies

Area	Technology
Reference platform	Android Automotive OS concepts (CarService, AudioService, Vehicle HAL)
Inter-process communication	Binder, AIDL
Projection	Android Auto / Apple CarPlay style (USB accessory mode, Wi-Fi)
Vehicle networks	CAN, UART (NMEA GNSS)
Audio hardware	I2S, TDM, A2B, DSP and amplifier
Diagramming	SVG / HTML page (included) and ReportLab (this report)

11. Validation Approach

Since this is a conceptual design, validation is by walkthrough. Each scenario in Section 7 is traced from input to output to confirm that every step has an owning component and a defined interface. The audio rules in Section 8 are checked against three conflict cases: navigation during media, a call during navigation, and media resume after a call. As a next step the design can be validated on the Android Automotive emulator by inspecting logcat output for audio focus changes and media session state transitions.

12. Limitations and Future Work

- The design is conceptual; no implementation or measurements are included.
- Security (secure boot, app sandboxing, authentication of projected devices) is not detailed.
- Functional safety (ISO 26262) and driver-distraction rules need dedicated analysis.
- Future work: build a media app prototype on the Android Automotive emulator, add multi-zone audio for rear-seat entertainment, and model interruptions in UML or Simulink.

13. Conclusion

The proposed five-layer architecture shows how media, navigation and projection can coexist on one IVI platform. Clear interfaces at each boundary, hardware abstraction through HALs, and centralised audio focus arbitration make behaviour predictable and the system maintainable. The scenarios show a complete path for user input to hardware output, and the interruption rules ensure that navigation prompts and phone calls are handled safely and that media resumes automatically.

14. References

- Android Open Source Project, Android Automotive OS documentation (source.android.com/docs/automotive).
- Android Developers, Audio focus and Media session guides (developer.android.com).
- Android Open Source Project, Vehicle HAL and Audio HAL documentation.
- Android Developers, Android for Cars (Android Auto and Car App Library) documentation.